
Election Management System

System Documentation

Soroti University Engineering Society

August 2026

Table of Contents

1. SUES Election Management System -- System Documentation

1 SUES Election Management System -- System Documentation

A complete technical reference for the Soroti University Engineering Society online voting platform.

Table of Contents

- [System Overview](#1-system-overview)
- [Architecture](#2-architecture)
- [Authentication & Eligibility](#3-authentication--eligibility)
- [Voter Registration & the System-Wide Register](#4-voter-registration--the-system-wide-register)
- [Election Lifecycle](#5-election-lifecycle)
- [The Voting Process (Atomic Receipt + Ballot)](#6-the-voting-process-atomic-receipt--ballot)
- [Results & Declaration Forms](#7-results--declaration-forms)
- [Audit Logging](#8-audit-logging)
- [Dashboard & Real-Time Subscriptions](#9-dashboard--real-time-subscriptions)
- [Firestore Data Model](#10-firestore-data-model)
- [Security Rules -- Full Reference](#11-security-rules--full-reference)
- [Roles & Permissions](#12-roles--permissions)
- [Client-Side Services](#13-client-side-services)
- [Deployment](#14-deployment)
- [Local Development & Emulator Suite](#15-local-development--emulator-suite)
- [Troubleshooting](#16-troubleshooting)

1. System Overview

The SUES Election Management System is a **serverless, client-side-driven** voting platform built on Firebase's free Spark plan. There are **no Cloud Functions** -- all sensitive operations (voting, role assignment) are enforced by Firestore security rules and atomic document operations on the client.

Key Design Decisions

Decision	Rationale
No Cloud Functions	Runs on Firebase Spark plan (free). All invariants enforced by security rules.
Atomic receipt + anonymous ballot	Double-vote prevention without a transactional backend. Ballot secrecy preserved.
System-wide register (`eligible_emails`)	A single CSV upload grants eligibility across all active elections.
Firestore Timestamps for election times	Rules compare `request.time` directly -- no string parsing, no timezone bugs.
Data-URL candidate photos	No Firebase Storage needed. Photos are compressed in-browser and stored as Base64 in the candidate document.
jspdf lazy-loaded	PDF generation never slows the initial page load.

Live URLs

Platform	URL
Firebase Hosting (primary)	https://suesvotingsystem.web.app

2. Architecture

```

????????????????????????????????????????????????????????????
?                               CLIENT (React + Vite)                ?
?                               ?                                    ?
?  LoginPage ??? VotePage ??? ResultsPage                          ?
?      ?                ?                ?                        ?
?      ?                ?                ?                        ?
?  ?????????????????????????????????????????????????????????? ?
?  ?          Services Layer          ?                            ?
?  ?  authService  voteService  electionSvc  ?                    ?
?  ?  rosterService  candidateSvc  auditSvc  ?                    ?
?  ?????????????????????????????????????????????????????????? ?
?      ?                ?                ?                        ?
?  ?????????????????????????????????????????????????????????? ?
?  ?          Firebase SDK (client)          ?                    ?
?  ?  Firestore reads/writes  ?  Auth          ?                    ?
?  ?????????????????????????????????????????????????????????? ?
????????????????????????????????????????????????????????????
?
?  ?????????????????????
?  FIREBASE      ?
?  ??????????   ?
?  Auth (Google) ?
?  Firestore     ?
?  Hosting       ?
?  ?????????????????????

```

What runs where

Concern	Where	Notes
Authentication	Firebase Auth (Google provider)	Popup-first, redirect fallback
Data storage	Cloud Firestore	All collections
Voting invariant enforcement	Firestore security rules	Atomic document-create locks
UI logic	React client	Permission guards, real-time subscriptions
PDF generation	Client-side (jspdf)	Lazy-loaded on demand
Role assignment	Client writes to `users/{uid}.role`	Security rules allow only Chairperson

3. Authentication & Eligibility

Sign-In Flow

```

User clicks "Sign in with Google"
?
?
Try signInWithPopup()
?

```

```

??????????
? Success ? Popup blocked / cancelled
?         ?
?         ?
finalizeSignIn()    signInWithRedirect()
?                  ?
?                  ?
Eligibility gate    getRedirectResult()
?                  ?
?                  ?
Profile bootstrap   finalizeSignIn()
?                  ?
?                  ?
Navigate to /vote    Navigate to /vote

```

Eligibility Gate (`finalizeSignIn`)

After Google authentication succeeds, the system checks:

- **Staff check**: Is the email in `eligible\_emails` with a role other than `VOTER`? (e.g., `ROLE\_CHAIRPERSON`, `ROLE\_SECRETARY`) -> **Staff: allowed in.**
- **Voter check**: If not staff, does at least one `voter\_roster` row exist with this email? -> **Voter: allowed in.**
- **Denied**: If neither -> **signed out immediately** with an error message.

A plain `VOTER` entry in `eligible\_emails` alone is **not sufficient** for login. The voter must also appear on at least one election roster. This prevents people who were uploaded to the register but never assigned to an election from accessing the system.

Account Chooser

The Google provider is configured with `prompt: "select\_account"`, forcing the account picker every time. This ensures users on shared devices can switch between Google accounts.

Multi-Layer Persistence

Firebase Auth is initialized with three persistence layers to survive browser storage partitioning:

```

indexedDBLocalPersistence (primary, survives most partitioning)
  ? fallback
browserLocalPersistence   (localStorage)
  ? fallback
browserSessionPersistence (sessionStorage)

```

Redirect State Guard

A `sessionStorage` flag (`sues\_auth\_redirect\_pending`) prevents:

- Stale redirect results from causing dead-ends
- Double-redirects from a second click overwriting the pending state
- The `getRedirectResult()` call is memoized so it's consumed exactly once

4. Voter Registration & the System-Wide Register

Two-Level Registration

The system has two registration layers:

Layer	Collection	Scope	Purpose
System-wide register	<code>`eligible_emails`</code>	All elections	Grants eligibility everywhere
Per-election roster	<code>`voter_roster`</code>	Single election	Tracks turnout for that election

How CSV Import Works

When an admin uploads a CSV file:

```
For each row (email, name):
  1. Create/update voter_roster/{voter_{electionId}_{email}}
     -> Sets electionId, voterEmail, voterName, hasVoted=false
     -> Preserves existing voting state if row already exists

  2. Create/update eligible_emails/{email}
     -> Sets email, role="VOTER", addedAt=now
     -> merge:true so existing staff roles are preserved
```

****Result**:** Every uploaded voter is eligible in ****every**** active election, not just the one they were imported for.

Register-Member Auto-Provision

When a register member visits the voting page for an election they don't have a roster row for:

```
VotePage loads
-> Checks rosterService.getMyRosterEntry() -> null
-> Checks rosterService.isOnRegister() -> true
-> Calls rosterService.ensureRosterRow() -> creates the row
-> Voter can now vote and appears in turnout metrics
```

This happens transparently -- the voter never sees the distinction.

Duplicate Prevention

- Roster row IDs are deterministic: ``voter_{electionId}_{email}`` -> same email can never appear twice per election
- Register entries use the email as the document ID -> naturally unique
- CSV import deduplicates within the file using a ``Set``

5. Election Lifecycle

Status Flow

```
draft ??? active ??? closed ??? published
? ?
??? (can reopen) ?????
```

Status	Who can see positions/candidat	Voting allowed	Results visible
`draft`	Admins only	No	No
`active`	Admins + voters (if on roster)	Yes (within time window)	Yes (real-time)
`closed`	Everyone	No	Yes
`published`	Everyone	No	Yes

Time Window Enforcement

Election times are stored as **Firestore Timestamps** (not ISO strings). The security rule ``electionOpen()`` compares directly:

```
election is active
AND (no startTime OR request.time >= startTime)
AND (no endTime OR request.time <= endTime)
```

The client also checks times before submitting a vote, providing friendly error messages ("Voting has not opened yet" / "Voting has closed") instead of generic permission errors.

Creating an Election (Admin Workflow)

- **Elections page** -> Click "New Election"
- Enter title, description, start/end times (stored as Timestamps)
- Status defaults to ``draft``
- **Positions page** -> Add positions (title, description, display order)
- **Candidates page** -> Add candidates per position (name, bio, optional photo)
- **Roster page** -> Import voters via CSV or add individually
- **Set status to `active`** -> Election is now live for voting

6. The Voting Process (Atomic Receipt + Ballot)

This is the core security mechanism. It replaces Cloud Functions with client-side atomic operations enforced by Firestore rules.

Step-by-Step

```
Voter clicks "Confirm Selection"
?
?
?? PRE-FLIGHT CHECKS (parallel, for UX) ??????????????
? 1. Does a receipt already exist? -> "Already voted" ?
? 2. Is the election active and within time window? ?
? 3. Is the voter on the roster or register? ?
????????????????????????????????????????????????????????
? (all pass)
?
?? STEP 1: CLAIM THE BALLOT ??????????????????????????
? Create vote_receipts/{email}__{election}__{position}?
? ?
? Document ID is deterministic -> a second attempt ?
? for the same position COLLIDES atomically. ?
? Firestore rejects it with "already-exists". ?
? ?
? Rules verify: ?
? - voterEmail matches authenticated user ?
? - nonce is a string >= 20 chars ?
```

```

?   - election is open (Timestamp check)           ?
?   - voter is on roster or register                ?
?   - position belongs to this election              ?
????????????????????????????????????????????????
?   ? (receipt created)
?
?? STEP 2: RECORD THE ANONYMOUS BALLOT ?????????????
?   Create votes/{nonce}                            ?
?                                                    ?
?   The nonce is a 40-char hex string (20 random bytes). ?
?   It's stored in the receipt but NOT linked to the ?
?   voter's identity. The vote document contains only: ?
?   - electionId, positionId, candidateId            ?
?   (NO createdAt: a timestamp on the ballot could ?
?   let an admin correlate it with the named receipt) ?
?                                                    ?
?   Rules verify:                                    ?
?   - A valid receipt exists for this election+position ?
?   - The candidate belongs to this position+election ?
?   - voteId (nonce) is >= 20 chars                  ?
????????????????????????????????????????????????
?   ? (ballot recorded)
?
?? STEP 3: ANONYMOUS AUDIT LOG ?????????????????????
?   Create audit_logs/{auto-id}                      ?
?   action: "CAST_VOTE"                              ?
?   entityType: "vote"                               ?
?   entityId: positionId (NOT the candidate)          ?
?   details: { electionId, positionId }               ?
?   (NO actorEmail, NO candidate -- ballot secrecy) ?
????????????????????????????????????????????????
?   ?
?
?? STEP 4: TURNOUT BOOKKEEPING ?????????????????????
?   Update voter_roster/{voter_{election}_{email}} ?
?   hasVoted: true                                    ?
?   votedPositions: arrayUnion(positionId)            ?
?   (best-effort, never blocks the vote)              ?
????????????????????????????????????????????????

```

Why This Is Secure

Threat	Mitigation
Double voting	Deterministic receipt ID -> atomic collision on second attempt
Concurrent race	Two voters submitting simultaneously: one receipt wins, the other gets `already-exists`
Ballot secrecy	Vote documents contain zero voter identity. Audit logs for CAST_VOTE have no `actorEmail`
Fake receipts	Rules verify the receipt's `voterEmail` matches the authenticated user
Fake candidates	Rules verify the candidate exists and belongs to the correct position+election
Voting outside window	Rules compare `request.time` against Firestore Timestamps
Non-eligible voting	Rules check `isOnRosterNow()` which includes both roster AND register

Error Handling

The client provides friendly error messages at each failure point:

Error	Meaning
"You have already voted for this position"	Receipt already exists (double-vote blocked)
"This election is not currently open for voting"	Election status is not `active`
"Voting for this election has not opened yet"	Current time < startTime
"Voting for this election has closed"	Current time > endTime
"You are not registered as an eligible voter"	Not on roster or register
"Ballot accepted but recording failed"	Receipt exists but vote write failed (edge case)

7. Results & Declaration Forms

Real-Time Results

The results page subscribes to Firestore in real time via `onSnapshot`. As votes are cast, tallies update instantly without page refresh.

Position Results Calculation

For each position:

- Filter candidates belonging to this position
- Sum votes per candidate from the `votes` collection
- Calculate percentages
- Sort by votes (descending)
- Check unopposed rule: if only 1 candidate, they need $\geq 51\%$ of votes cast to be "affirmed"

Unopposed 51% Rule

A lone candidate must receive at least 51% of votes cast for the position. This ensures voter confidence -- if voters reject an unopposed candidate (by casting blank/spoiler ballots in a system that allows it), the nomination is not affirmed.

PDF Exports

Two PDF formats are available:

1. Standard Results Report (`downloadPdf`)

- SUES logo + header
- Per-position bar charts with vote counts and percentages
- Turnout summary (eligible voters, voters who voted, turnout %)
- Anonymous ballot disclaimer

2. Declaration of Results (`downloadDeclarationForm`)

- **SUES-branded header**: Dark navy band with gold accent stripe, SUES logo, "SOROTI UNIVERSITY / ENGINEERING SOCIETY / Electoral Commission"
- **Election details**: Title, poll opened/closed times, date declared
- **Candidates declared elected**: Green header bar listing winners per position
- **Per-position result tables**: Navy accent bars, alternating row shading, winner bold highlight, gold-bordered declaration line

- ****Summary of returns****: Total votes, registered voters, participants, turnout, invalid ballots note
- ****Certification statement****: "I, the Returning Officer, hereby certify..."
- ****Four signature lines****: Returning Officer, Candidate/Representative, Observer 1, Observer 2
- ****Footer****: Generation timestamp

Public Results Access

Results are accessible to ****all authenticated users**** (including voters) at `/results``. This route is outside the admin layout, so voters see a clean interface with a "Back to Voting" link.

The Firestore rules allow this by permitting all signed-in users to read:

- ``votes`` (anonymous -- no voter identity)
- ``voter_roster`` (semi-public in a university society context)

8. Audit Logging

How It Works

Every mutation in the system calls ``auditService.log()`` as a ****best-effort, non-blocking**** operation. If the audit write fails, the main operation is never affected.

Logged Actions

Action	Triggered by	Contains voter identity?
<code>`CAST_VOTE`</code>	<code>`voteService.submitVote()`</code>	**No** (ballot secrecy)
<code>`ELECTION_CREATED`</code>	<code>`electionService.createElection()`</code>	Yes (actor email)
<code>`ELECTION_UPDATED`</code>	<code>`electionService.updateElection()`</code>	Yes
<code>`ELECTION_DELETED`</code>	<code>`electionService.deleteElection()`</code>	Yes
<code>`POSITION_CREATED`</code>	<code>`electionService.createPosition()`</code>	Yes
<code>`POSITION_UPDATED`</code>	<code>`electionService.updatePosition()`</code>	Yes
<code>`POSITION_DELETED`</code>	<code>`electionService.deletePosition()`</code>	Yes
<code>`CANDIDATE_CREATED`</code>	<code>`candidateService.createCandidate()`</code>	Yes
<code>`CANDIDATE_UPDATED`</code>	<code>`candidateService.updateCandidate()`</code>	Yes
<code>`CANDIDATE_DELETED`</code>	<code>`candidateService.deleteCandidate()`</code>	Yes
<code>`ROSTER_IMPORTED`</code>	<code>`rosterService.bulkUploadRoster()`</code>	Yes
<code>`VOTER_ADDED`</code>	<code>`rosterService.addVoter()`</code>	Yes
<code>`VOTER_REMOVED`</code>	<code>`rosterService.removeVoter()`</code>	Yes
<code>`USER_ROLE_UPDATED`</code>	<code>`authService.updateUserRole()`</code>	Yes

Audit Log Document Structure

```
{
  "action": "CAST_VOTE",
  "entityType": "vote",
  "entityId": "position_def",    // the position -- the candidate is never recorded
  "details": {
    "electionId": "election_xyz",
    "positionId": "position_def"
  },
}
```

```
"createdAt": "Timestamp",
"actorEmail": "" // omitted for CAST_VOTE
}
```

Real-Time Display

The Audit Logs page (`/admin/audit`) subscribes via `onSnapshot` -- entries appear the instant they're written. Only the Administrator or Chairperson can read audit logs (enforced by security rules).

9. Dashboard & Real-Time Subscriptions

Election-Scoped Dashboard

The admin dashboard is **election-scoped** -- the Chairperson selects an election from a dropdown, and all metrics update for that specific election.

Subscriptions Used

Subscription	Collection	Updates
<code>`electionService.subscribeToElections()</code>	<code>`elections`</code>	Election list
<code>`candidateService.subscribeToCandidates(electionId)`</code>	<code>`candidates`</code>	Candidate count, photos
<code>`rosterService.subscribeToRoster(electionId)`</code>	<code>`voter_roster`</code>	Eligible voters, voters who voted
<code>`voteService.subscribeToVotes(electionId)`</code>	<code>`votes`</code>	Live vote tallies

All subscriptions use Firestore's `onSnapshot` for real-time updates. The dashboard shows:

- Total positions
- Total candidates
- Eligible voters
- Voters who have voted
- Turnout percentage

10. Firestore Data Model

Collections Overview

```
firestore/
??? users/{uid}           # User profiles + roles
??? elections/{electionId} # Election definitions
??? positions/{positionId} # Positions within elections
??? candidates/{candidateId} # Candidates per position
??? voter_roster/{voter_{eid}_{email}} # Per-election voter rows
??? eligible_emails/{email} # System-wide register
??? vote_receipts/{email}_{eid}_{pid} # Once-only ballot locks
??? votes/{nonce}         # Anonymous ballot records
??? audit_logs/{auto-id}  # Audit trail
```

Document Schemas

``users/{uid}``

```
{
```

```
"email": "chair.sues@sun.ac.ug",
"fullName": "John Doe",
"role": "ROLE_CHAIRPERSON",
"createdAt": "Timestamp",
"updatedAt": "Timestamp"
}
```

`elections/{electionId}`

```
{
  "title": "SUES Guild Elections 2026",
  "description": "Annual guild elections",
  "status": "active",           // draft | active | closed | published
  "startTime": "Timestamp",    // Firestore Timestamp, not ISO string
  "endTime": "Timestamp",
  "resultsPublished": false
}
```

`positions/{positionId}`

```
{
  "electionId": "election_xyz",
  "title": "President",
  "description": "Society president",
  "maxVotes": 1,
  "displayOrder": 1
}
```

`candidates/{candidateId}`

```
{
  "electionId": "election_xyz",
  "positionId": "position_def",
  "name": "Jane Smith",
  "bio": "Third-year Electrical Engineering student",
  "photoUrl": "data:image/jpeg;base64,...", // compressed in-browser
  "displayOrder": 1
}
```

`voter\_roster/voter\_{electionId}\_{email}`

```
{
  "electionId": "election_xyz",
  "voterEmail": "student@sun.ac.ug",
  "voterName": "Student Name",
  "hasVoted": true,
  "votedPositions": ["position_def", "position_ghi"]
}
```

`eligible\_emails/{email}`

```
{
  "email": "student@sun.ac.ug",
  "role": "VOTER",
  "addedAt": "Timestamp"
}
```

`vote\_receipts/{email}\_{electionId}\_{positionId}`

```
{
  "voterEmail": "student@sun.ac.ug",
  "electionId": "election_xyz",
  "positionId": "position_def",
}
```

```
"nonce": "a1b2c3d4e5f6...", // 40-char hex string
"createdAt": "Timestamp"
}
```

``votes/{nonce}``

```
{
  "electionId": "election_xyz",
  "positionId": "position_def",
  "candidateId": "candidate_abc"
  // NO voter identity and NO timestamp -- ballot is anonymous
}
```

``audit_logs/{auto-id}``

```
{
  "action": "CAST_VOTE",
  "entityType": "vote",
  "entityId": "position_def", // the position, never the candidate
  "details": { "electionId": "...", "positionId": "..." },
  "createdAt": "Timestamp"
  // actorEmail omitted for CAST_VOTE; choice never recorded
}
```

11. Security Rules -- Full Reference

Helper Functions

Function	Purpose
<code>`isSignedIn()`</code>	Checks <code>`request.auth != null`</code>
<code>`userEmail()`</code>	Returns the authenticated user's email
<code>`myRole()`</code>	Reads <code>`users/{uid}.role`</code> with <code>`exists()`</code> guard for new accounts
<code>`isAdminRole()`</code>	Administrator, Chairperson, Secretary, or Assistant
<code>`isChairperson()`</code>	Administrator or Chairperson
<code>`isElectionManager()`</code>	Administrator, Chairperson, or Secretary
<code>`validRole(role)`</code>	Role must be one of the 5 valid roles: Administrator, Chairperson, Secretary, Assistant, Voter
<code>`electionStatus(electionId)`</code>	Reads the election's <code>`status`</code> field
<code>`electionOpen(electionId)`</code>	Checks <code>status=active</code> AND time window
<code>`isRegisterMember()`</code>	Checks <code>`eligible_emails/{email}`</code> exists
<code>`isOnRosterNow(electionId)`</code>	Roster row OR register member
<code>`receiptId(electionId, positionId)`</code>	Builds the deterministic receipt ID

Per-Collection Rules

Collection	Read	Write
`users`	Own profile + election managers	Self (name only) / Admin + Chair (roles)
`elections`	Any signed-in user	Election managers
`positions`	Admins + active/closed/published elections	Election managers
`candidates`	Admins + active/closed/published elections	Election managers
`voter_roster`	Any signed-in user	Managers (full) / voters (own turnout flags, additive-only)
`eligible_emails`	Own entry + election managers	Election managers
`vote_receipts`	Own receipts	Create-once (atomic lock)
`votes`	Any signed-in user	Create-once (nonce-keyed, no timestamp)
`audit_logs`	Admin + Chairperson only	Create with action validation

Key Invariants Enforced by Rules

- ****Double-vote prevention****: `vote\_receipts` document ID is deterministic -> second create attempt collides atomically
- ****Ballot secrecy****: `votes` documents contain no voter identity and no timestamp; `CAST\_VOTE` audit logs have no `actorEmail` and record the position, never the candidate
- ****Time enforcement****: `electionOpen()` compares `request.time` against Firestore Timestamps
- ****Eligibility****: `isOnRosterNow()` checks both roster AND register
- ****Role integrity****: Profile creation must mirror the register entry's role, restricted to the 5 valid roles via `validRole()`
- ****Turnout flags****: Voters can only flip their own `hasVoted`/`votedPositions`, and only additively (never remove)

12. Roles & Permissions

Role Matrix

Permission	Administrator	Chairperson	Secretary	Assistant	Voter
VIEW_DASHBOARD	[x]	[x]	[x]	[x]	--
MANAGE_ELECTIONS	[x]	[x]	[x]	--	--
MANAGE_CANDIDATES	[x]	[x]	[x]	--	--
MANAGE_ROSTER	[x]	[x]	[x]	[x]	--
VIEW_RESULTS	[x]	[x]	[x]	--	[x]
MANAGE_USERS	[x]	[x]	--	--	--
VIEW_AUDIT_LOGS	[x]	[x]	--	--	--
VOTE	[x]	[x]	[x]	[x]	[x]
PUBLISH_RESULTS	[x]	[x]	--	--	--

The **Administrator** is the system owner with full access and is **exempt from the maintenance lock** (`system_config/maintenance`). During maintenance, only the Administrator may sign in.

How Roles Are Assigned

- **Seeded staff**: Administrator/Chairperson/Secretary emails are pre-seeded in `eligible_emails` with their admin role before any sign-in
- **First sign-in**: `finalizeSignIn()` reads the register entry's role and writes it to `users/{uid}.role`
- **Runtime changes**: Only the Administrator or Chairperson can update roles via the User Management page

How Roles Are Enforced

- **UI**: `PermissionGuard` component checks `permissions.includes(permission)` and redirects unauthorized users
- **Server**: Firestore security rules read `users/{uid}.role` via the `myRole()` helper function
- **Navigation**: `AdminLayout` filters sidebar items by the user's permissions

13. Client-Side Services

Service Map

```
authService
  ??? signInWithGoogle()      -> popup/redirect + finalizeSignIn
  ??? resolveRedirectSignIn() -> memoized redirect result handler
  ??? finalizeSignIn()       -> eligibility gate + profile bootstrap
  ??? getUserProfile()       -> reads users/{uid}
  ??? getAllProfiles()       -> reads all users (admin)
  ??? updateUserRole()       -> writes users/{uid}.role
  ??? signOut()

electionService
  ??? subscribeToElections()  -> real-time election list
  ??? getElections()         -> one-shot election list
  ??? getPositions(electionId) -> positions for an election
  ??? createElection()        -> + audit log
  ??? updateElection()        -> + audit log
  ??? deleteElection()        -> + audit log
  ??? createPosition()        -> + audit log
  ??? updatePosition()        -> + audit log
  ??? deletePosition()        -> + audit log

candidateService
  ??? getCandidates(electionId)
  ??? subscribeToCandidates(electionId) -> real-time
  ??? createCandidate()         -> + audit log
  ??? updateCandidate()         -> + audit log
  ??? deleteCandidate()         -> + audit log
  ??? uploadCandidatePhoto()     -> in-browser JPEG compression

rosterService
  ??? subscribeToRoster(electionId) -> real-time
  ??? getRoster(electionId)
  ??? addVoter()                 -> + register entry + audit log
  ??? removeVoter()              -> + audit log
  ??? isOnRoster(electionId, email)
  ??? getMyRosterEntry(electionId, email) -> voter's own row
```

```

??? isOnRegister(email)      -> checks eligible_emails
??? ensureRosterRow(electionId, email)  -> auto-provision
??? bulkUploadRoster()       -> CSV import + register + audit log

```

voteService

```

??? submitVote(electionId, positionId, candidateId)
?      -> pre-flight checks
?      -> create receipt (atomic lock)
?      -> create anonymous ballot
?      -> anonymous audit log
?      -> turnout bookkeeping
??? subscribeToVotes(electionId) -> real-time
??? getVotes(electionId)

```

auditService

```

??? log(action, entityType, entityId, details?) -> best-effort write
??? getAuditLogs()                             -> one-shot (last 100)
??? subscribeToAuditLogs()                     -> real-time (last 100)

```

14. Deployment

Firebase Hosting

```

# 1. Hide .env (contains dev/emulator config)
Move-Item .env .env.dev-backup

# 2. Clear Vite cache
Remove-Item -Recurse -Force node_modules\.vite

# 3. Build
npm run build

# 4. Deploy rules + hosting
$env:FIREBASE_SKIP_FRAMEWORK_SETUP = "1"
npx firebase deploy --only firestore:rules,hosting --project suesvotingsystem

# 5. Restore .env
Move-Item .env.dev-backup .env

```

Why .env Must Be Hidden

The `.env` file contains:

- `VITE\_USE\_EMULATORS=true` -- would try to connect to local emulators in production
- old project Firebase config -- wrong project

The production build uses the hardcoded `suesvotingsystem` fallback defaults when env vars are absent.

GitHub Repos

> Development now happens **only** on the fork. No further PRs are pushed to > `AbelEaX/sues` (upstream); the `origin` remote pointing there has been removed.

Repository	Branch	Purpose
`noahotim/sues`	`fix/vote-roster-eligibility`	Fork -- primary development repo
`noahotim/sues-deploy`	`main`	Deployment copy

15. Local Development & Emulator Suite

Quick Start

```
powershell -ExecutionPolicy Bypass -File start-demo.ps1
```

This single command:

- Starts Firebase emulators (Auth:9099, Firestore:8080, Storage:9199, UI:4000)
- Wipes stale collections
- Seeds elections, positions, candidates, roster, and register entries
- Starts the Vite dev server on port 5173

Demo Accounts

Role	Email	Password
Chairperson	chair.sues@sun.ac.ug	sues2026
Secretary	secretary.sues@sun.ac.ug	sues2026
Voter	apio.samson@sun.ac.ug	sues2026

Test Scripts

Script	Purpose
`functions/verify-demo.mjs`	Full lifecycle verification (9 checks)
`functions/test-string-times.mjs`	Regression test for Timestamp-based voting
`functions/demo-full-process.mjs`	End-to-end: seed -> vote -> verify -> results
`scripts/fb-backfill-times.cjs`	Production backfill: ISO strings -> Timestamps

16. Troubleshooting

"Vote could not be recorded"

****Cause**:** The election times are stored as ISO strings instead of Firestore Timestamps, or the voter is not on the roster/register.

****Fix**:** Run `node scripts/fb-backfill-times.cjs` to convert existing elections. Ensure the voter's email is in `eligible\_emails` or on a `voter\_roster` row.

Results page shows no data for voters

****Cause**:** Firestore security rules restrict `votes` and `voter\_roster` reads to admins only.

****Fix**:** Ensure `votes` read rule is `allow read: if isSignedIn()` and `voter\_roster` read rule is `allow read: if isSignedIn()`. Deploy rules: `npx firebase deploy --only firestore:rules --project suesvotingsystem`.

Login bounces back to login page

****Cause**:** The user's email is not on the register or any roster. The eligibility gate signs them out immediately.

****Fix**:** Add the email to `eligible\_emails` (for staff) or import them into a `voter\_roster` via CSV (for voters).

Redirect sign-in shows "missing-initial-state"

****Cause**:** Browser storage partitioning or a stale redirect.

****Fix**:** The system handles this gracefully -- it clears the redirect state and returns to a clean login screen. The user should try signing in again.

Vercel deployment shows wrong Firebase project

****Cause**:** Vite cache (`node\_modules/.vite`) cached the old `.env` values.

****Fix**:** `Remove-Item -Recurse -Force node\_modules\.vite` before building.

Documentation generated for the SUES Election Management System. *Last updated: August 2026.*